

5.1 Formal Test Plan: Reliabuy Real Estate Platform

1. Project and Application Overview

Reliabuy is a full-stack real estate marketplace designed to connect buyers and sellers with a low-friction, high-performance experience. Built on a modern stack (React/TypeScript frontend, Node.js/Express backend, PostgreSQL database), the application allows users to browse, search, make offers, and manage property listings. Key architectural pillars include raw, parameterized SQL queries for database interactions, JWT-based authentication, and a layered backend routing pattern. The frontend emphasizes an optimistic, snappy UI with features like AI-generated property descriptions and debounced search.

2. Testing Scope

In Scope:

- **Frontend UI/UX:** Core components including `PropertySearch.tsx`, `MyListings.tsx` (optimistic updates), `PropertyDetail.tsx` (state toggles/ownership checks), and `SavedProperties.tsx`.
- **Backend APIs:** All critical endpoints including `/search`, `/my-listings`, `/saved`, CRUD operations for properties, and offer submissions.
- **Database:** Validation of raw SQL queries, cascading deletes, and strict prevention of SQL injection via parameterized queries.
- **Security:** JWT verification middleware, basic input validation, and HTTP security headers.
- **Performance:** System stability under load testing up to a maximum of 1,000 Concurrent Virtual Users (VUs).
- **Usability:** Assessment of core user flows, navigation simplicity, and content readability.

Out of Scope:

- Third-party payment gateway integrations (not currently implemented).
 - Advanced data manipulation features (Filtering) and live Chat systems (identified as gaps, scheduled for future development phases).
 - Hardware-level infrastructure testing beyond the local loopback/Node.js backend configuration.
-

3. Testing Types and Techniques

Testing Type	Technique / Focus
Functional/API Testing	Validating HTTP status codes (200, 400, 401, 403, 404, 500) and Express controller logic. Ensuring static routes successfully bypass dynamic <code>/:id</code> conflicts.
Integration Testing	Verifying the Route → Controller → Service pipeline and ensuring database transactions return correct JSON payloads.
Performance Testing	Executing Load (200 VU), Stress (500 VU), and Peak (1000 VU) scenarios to monitor average latency, throughput (RPS), and connection pool stability.
Security Testing	Automated scanning (DAST) for header vulnerabilities combined with manual penetration testing for SQLi, XSS, and authorization bypasses.
Usability Testing	Task-based qualitative testing focusing on perceived ease of use, navigation friction, and the impact of AI-generated descriptions.

4. Roles and Responsibilities

- Web Developer
 - Test Developer
-

6. Entry and Exit Criteria

Entry Criteria:

- Backend REST API endpoints are functionally complete and deployed to the test environment.
- Frontend components are merged into the main testing branch.
- Test data (dummy users, properties, JWT tokens) is populated in the PostgreSQL database.

Exit Criteria:

- Majority of planned functional and integration test cases executed.
- Zero critical or high-severity functional bugs open.
- System successfully sustains 1,000 VUs with a success rate of >98%.
- All identified medium/high security vulnerabilities (e.g., missing CSP headers) are either patched or have a documented mitigation plan.

7. Tools and Environments

- **Test Environment:** System Under Test (SUT) configured on a local loopback network targeting the Node.js Backend.
 - **Database Tooling:** PostgreSQL CLI / pgAdmin for verifying raw query state.
 - **API / Functional Testing:** Playwright Testing Library for frontend and backend component state.
 - **Performance Testing:** k6 framework utilizing dynamic runtime CLI overrides.
 - **Security Testing:** OWASP ZAP for automated vulnerability scanning.
-